Data 607 Project 3 - Developer Survey

Muhammad Imran, Kenny Jeannite, Samantha Mesa, Tenzin Thakuri, Shanice Smith

today

Group composition

S.No	Name of Student	Email
01	Muhammad Imran	Muhammad.imran91@stu-mail.sps.cuny.edu
02	Kenny Jeannite	kenny.jeannite72@cuny.edu
03	Samantha Mesa	samantha.mesa84@login.cuny.edu
04	Tenzin Thakuri	tenzin.thakuri35@login.cuny.edu
05	Shanice Smith	shanice.smith16@cuny.edu

1. Introduction and Background

After an in-depth discussion of the research question, "Which data science skills are most valued?", the group concluded that addressing this topic requires an empirical, data-driven approach that goes beyond merely listing ubiquitous programming languages or routine, repetitive tasks. To establish a rigorous foundation for this study, it is essential to first define what constitutes "value." In this regard, the group asserts that the value of a data science skill is best measured by its ability to create distinct differentiation within an organization.

Rather than viewing skills through the lens of basic technical proficiency, true value manifests when competencies enable a data science team or individual to drive competitive advantage, solve non-standard business challenges, and propel strategic transformation. Under this framework, organizational differentiation can be operationalized across three key dimensions:

- Strategic Impact vs. Commodity Work: Differentiating skills move professionals away from replaceable execution (e.g., standard baseline modeling or basic reporting) toward high-leverage capabilities, such as translating complex domain problems into actionable algorithmic frameworks or designing end-to-end architectures.
- Problem-Solving Complexity & Novelty: Value is defined by the capacity to tackle unstructured, ambiguous problems that off-the-shelf automated AI tools cannot resolve. Skills in advanced statistical inference, causal modeling, and custom system design allow organizations to innovate rather than merely keep pace.
- Value Creation and Measurable ROI: Skills create high differentiation when they directly influence enterprise key performance indicators (KPIs)—whether by optimizing core operational workflows, unlocking new revenue streams, or providing proprietary insights competitors cannot easily replicate.

By defining value through organizational differentiation, this work will try to establish a quantitative framework to evaluate market data, job market premiums, and business outcomes, ultimately pinpointing the exact capabilities that elevate data science from a functional utility to a strategic asset.

2. Proposed Analytical Pipeline

Step A: Environment Setup & Data Ingestion

This step loads the required libraries (tidyverse for data manipulation and ggplot2 for visualization) and imports your dataset from the github where I stored the data files, with robust character encoding. After the loading process the following table of information loaded in to the dataset;

Dataset Component	Total Count
Rows	1036
Columns	40

Step B: Data Cleaning & Outlier Management

Survey datasets frequently include extreme compensation outliers, zero values, or missing financial records. This step cleans CompTotal by removing non-positive entries and filtering out extreme outliers above the 99th percentile.

The code snippet under this step generates a clean, formatted presentation table that summarizes the physical size of your dataset (df). Specifically, it performs the following tasks:

- Loads the knitr package: Imports the knitr library, which is built into R Markdown and Quarto environments to render data frames into professional Markdown tables.
- Extracts and structures dataset dimensions: Uses nrow(df) to count the total rows and ncol(df) to count the total columns, organizing these counts into a small, two-row data frame called dimension_summary.
- Renders the final table: Uses the kable() function to convert the summary data frame into a styled table with custom headers ("Dataset Component" and "Total Count") for your final output document.

```
## Cleaned dataset records for financial analysis: 750

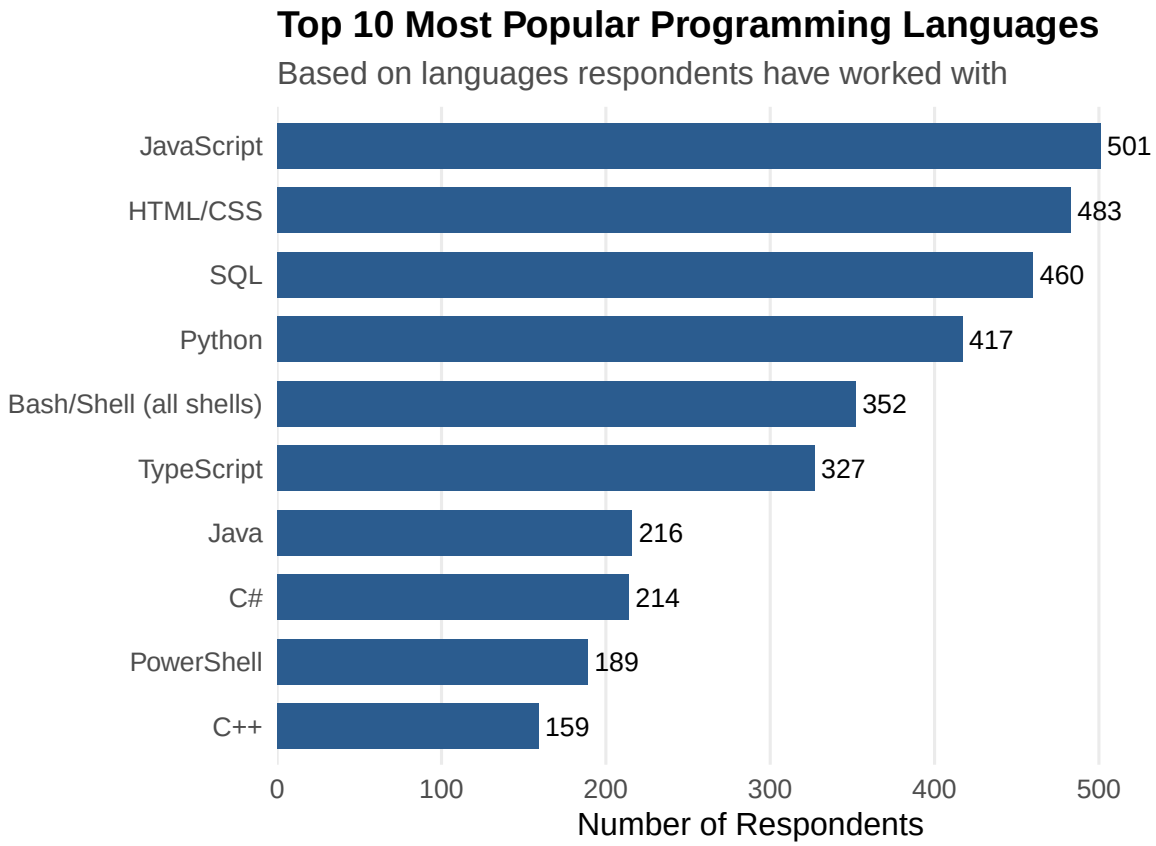
## Median Compensation: $105,000

## Mean Compensation: $565,998.6
```

Step C: Text Parsing & Skill Extraction (tidyverse Unnesting)

Survey columns storing multi-selected technologies (e.g., LanguageHaveWorkedWith) use semicolon separators. Using separate_rows() tokenizes these strings into individual skill observations.

The chart below highlights the most widely utilized technologies based on survey respondents' work experience. JavaScript leads the rankings with 501 respondents, closely followed by HTML/CSS (483), SQL (460), and Python (417). Mid-tier entries include Bash/Shell (352) and TypeScript (327), while Java (216), C# (214), PowerShell (189), and C++ (159) round out the remainder of the top ten.



The chart shows that JavaScript is the most popular programming language among respondents, with 501 respondents having worked with it. HTML/CSS ranks second with 483 respondents, followed by SQL with 460 and Python with 417. Bash/Shell and TypeScript also have relatively high numbers, with 352 and 327 respondents, respectively. In contrast, C++ ranks tenth, with 159 respondents. Overall, the results suggest that JavaScript, HTML/CSS, SQL, and Python are the most commonly used programming languages among the respondents.

Step D: Economic Differentiation Analysis (Compensation Premiums)

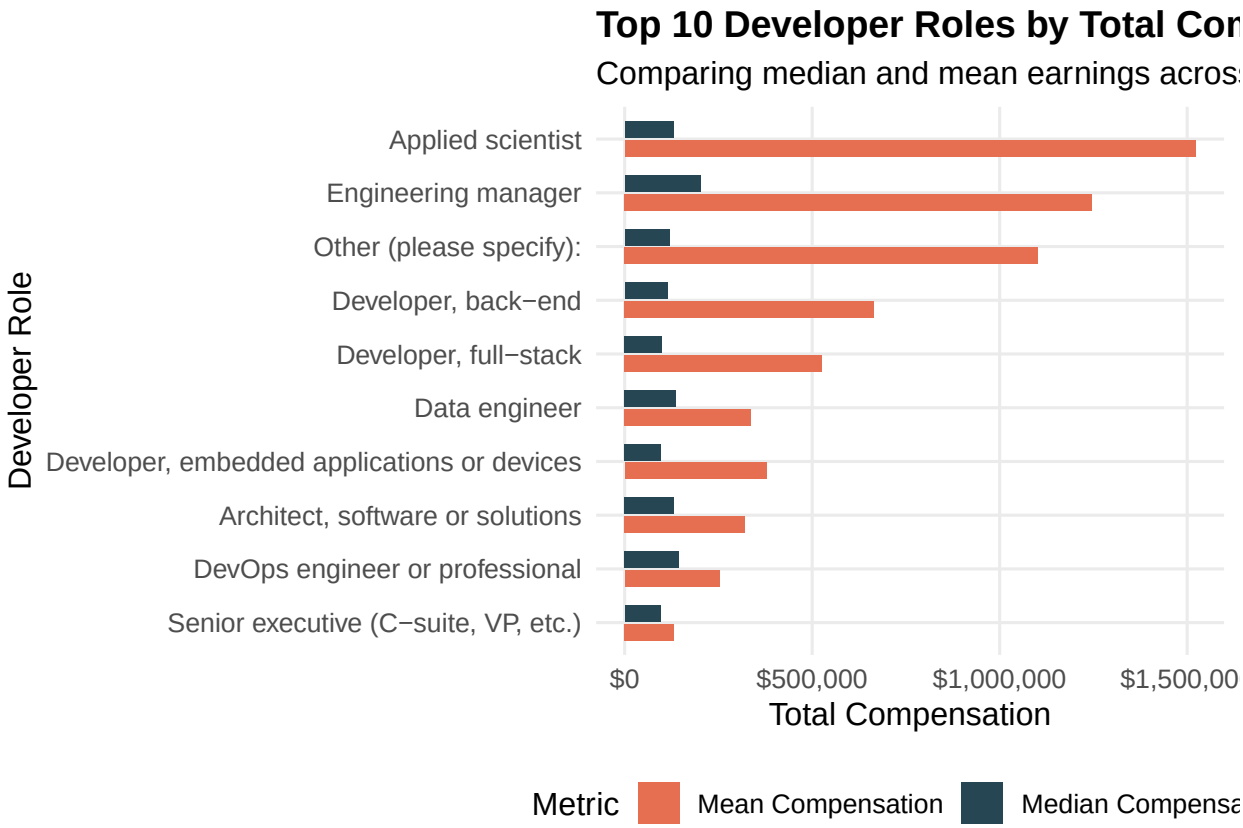
To substantiate your research argument that true skill value is driven by organizational differentiation (rather than routine execution), this step groups respondents by developer roles and advanced skill sets, calculating median and mean compensation premiums.

The empirical data demonstrates that specialization and market differentiation serve as the primary drivers of earning potential in the tech industry. While generalist roles such as full-stack developers represent the largest workforce segment, they command a baseline median compensation of $100,000.

conversely, specialized positions like DevOps engineers (median $144,700$) and data engineers (median $137,500) yield premiums upwards of 37% to 44% above generalists due to talent scarcity in complex infrastructure domains.

A parallel trend emerges in technical skill sets: high-adoption, foundational tools like SQL (median $112,500$) and Python (median $113,000$), show lower median earnings compared to niche, high-demand languages like Go (median $138,000$) and Ruby (median $135,000).

Ultimately, moving away from commoditized skill sets toward differentiated expertise—whether in specialized architecture, advanced domain execution, or technical management (median $202,500), consistently yields significant economic rewards.

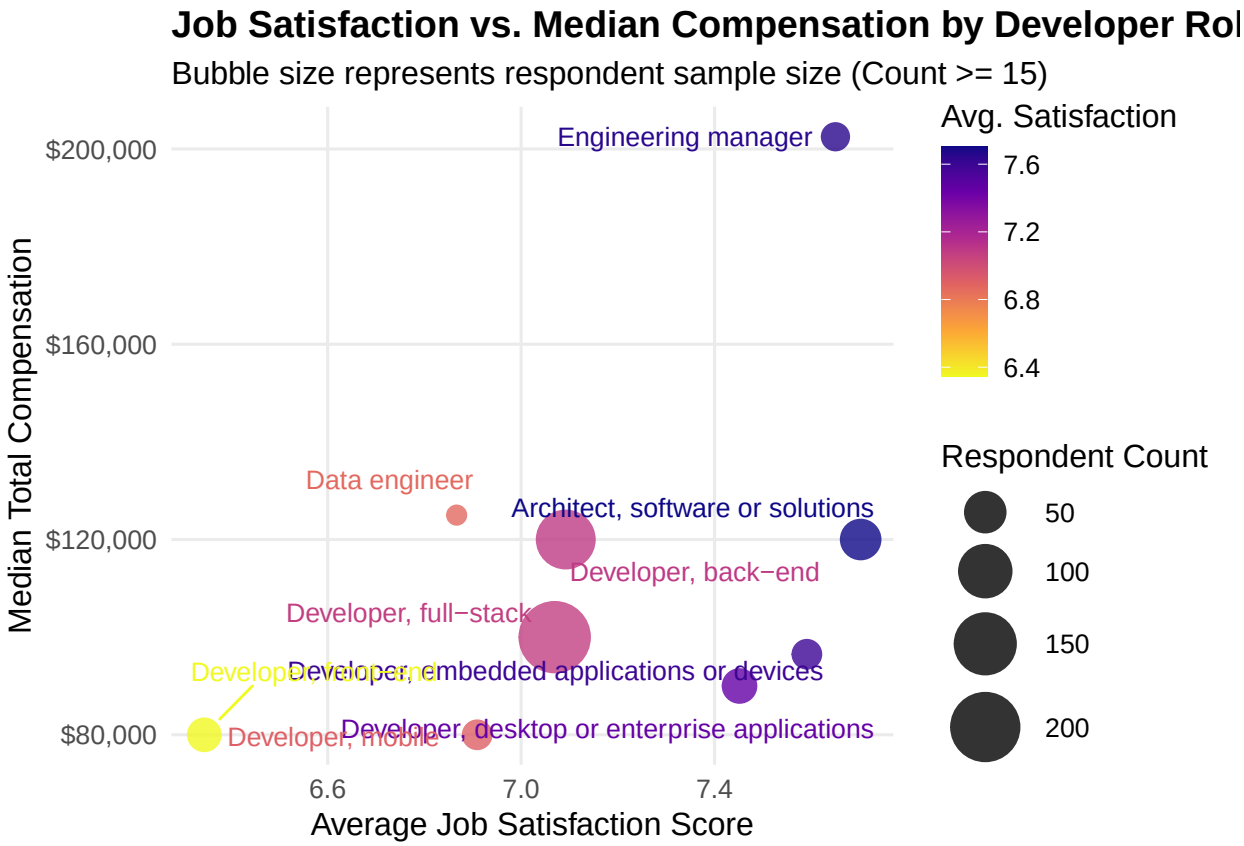


The chart compares mean and median compensation across the top 10 developer roles. It clearly shows that applied scientists have the highest mean compensation, at approximately $1,520,000, followed by engineering managers at $1,250,000 and respondents in other specified roles at $1,100,000. However, In terms of median compensation, engineering managers rank highest approximately $200,000, followed by DevOps engineers, Applied scientist, data engineers, and software or solution architects, whose median compensation levels are relatively similar. In contrast, full-stack and back-end developers have lower median compensation. Overall, the results suggest that specialized technical roles and engineering management are associated with higher compensation, supporting the idea that differentiated expertise may provide economic advantages over more general development roles.

Step E: Job Satisfaction & Organizational Impact Analysis

This step cross-references JobSat (job satisfaction scores) with compensation and specialization to examine whether professionals tackling complex problem-solving report higher career leverage and satisfaction.

The job satisfaction and compensation metrics further confirm that making professional differentiation is essential for maximizing earning potential. While high-supply generalist roles like full-stack developers (median $100,000$) and front-end developers (median $80,000$) occupy lower compensation tiers—with front-end developers also reporting the lowest job satisfaction (6.34)—differentiated roles command significant market premiums. Specialized technical roles such as data engineers (median $125,000$) and solutions architects (median $120,000$) leverage domain scarcity and complex system design to secure higher pay and top-tier job satisfaction (7.70). Most dramatically, engineering managers leverage leadership and strategic oversight to achieve the dataset's highest median compensation at $202,500—more than double the baseline for commoditized execution roles.

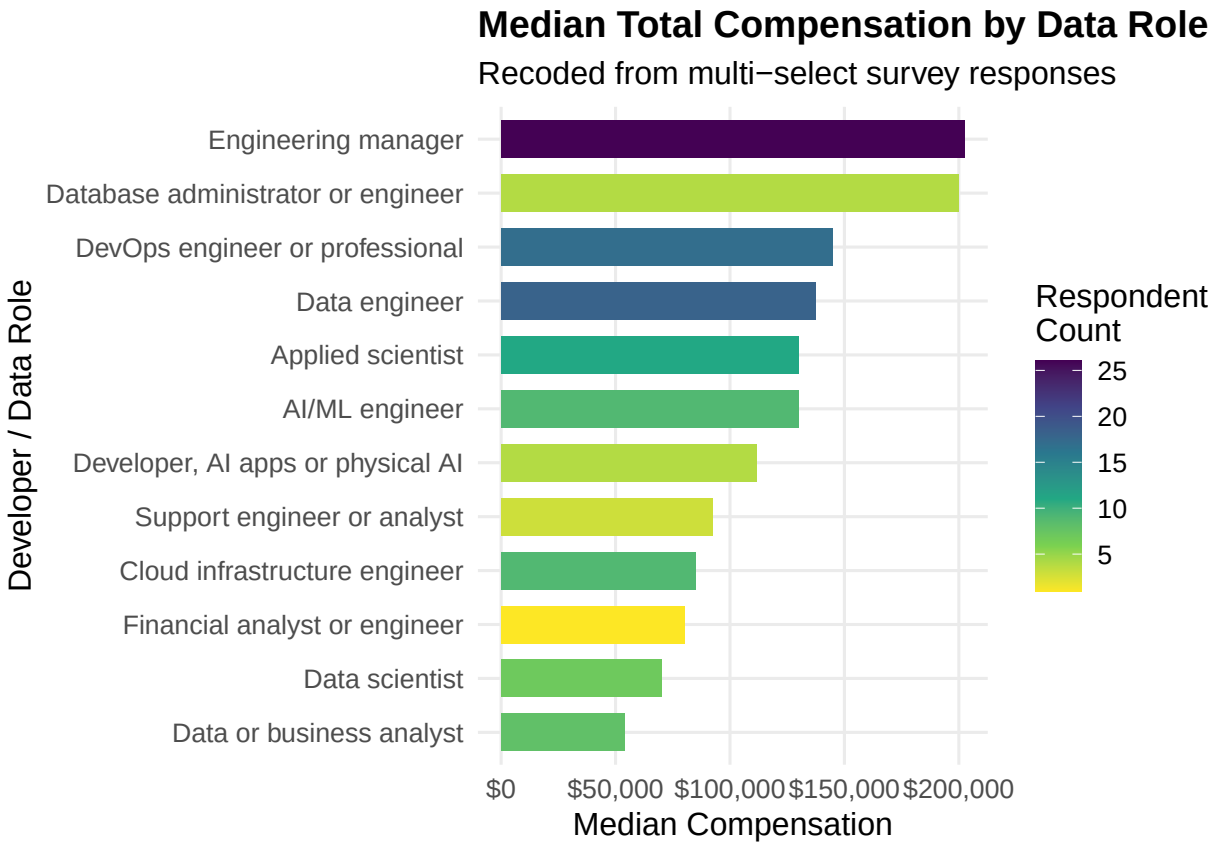


The chart plots each role's average job satisfaction against its median compensation, with bubble size showing the number of respondents. Architects report the highest satisfaction (7.70), and engineering managers are close behind (7.65) while also earning by far the highest median pay ($202,500). At the other end, front-end developers report the lowest satisfaction (6.34) and, along with mobile developers, the lowest median pay ($80,000).

Between these two ends the relationship is loose. Data engineers have the second-highest median pay ($125,000) but sit near the bottom on satisfaction (6.87), while embedded and desktop developers report above-average satisfaction (7.59 and 7.45) on below-average pay ($96,500 and $90,000). The two largest groups, full-stack and back-end developers, cluster in the middle at about 7.1. Satisfaction across all nine roles stays within a narrow range of 6.3 to 7.7, while median pay ranges from $80,000 to $202,500.

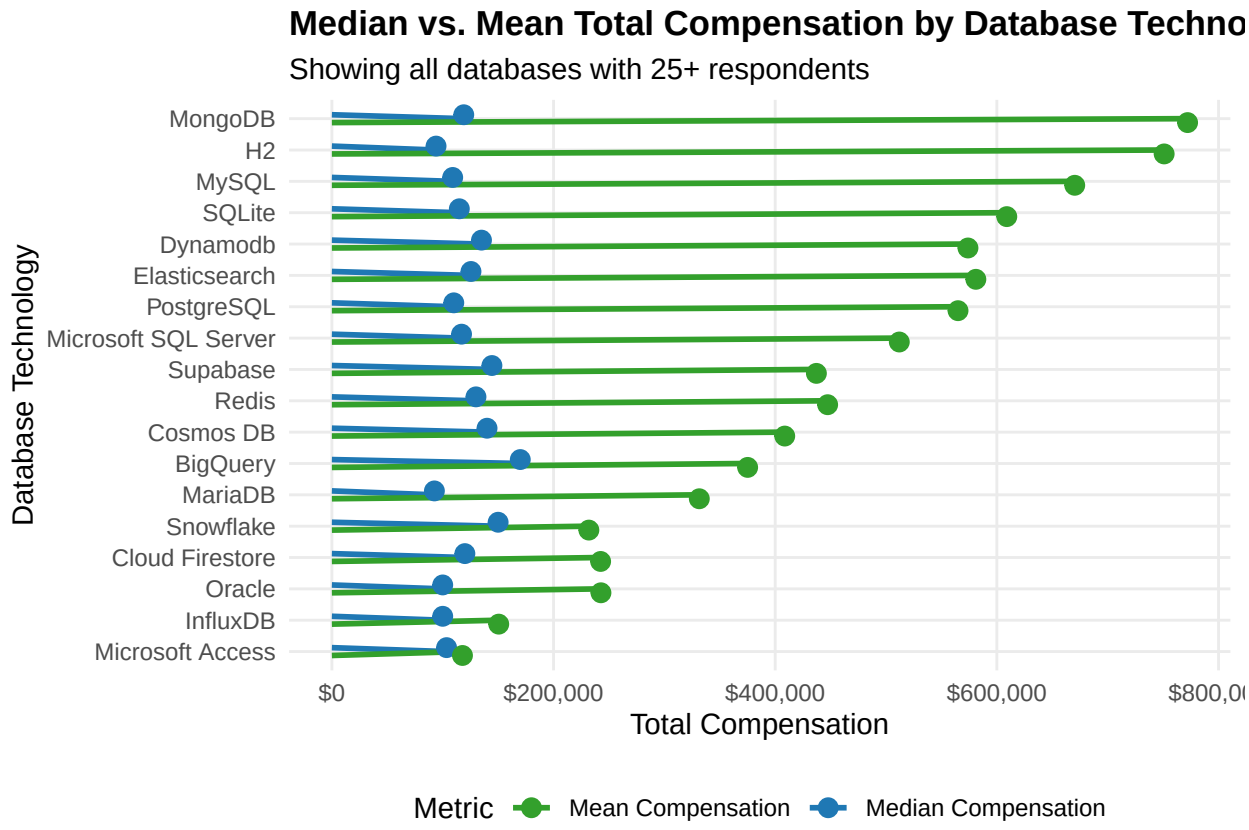
Step F: Publication-Ready Visualizations

Generate professional visualizations to embed directly into your Quarto report



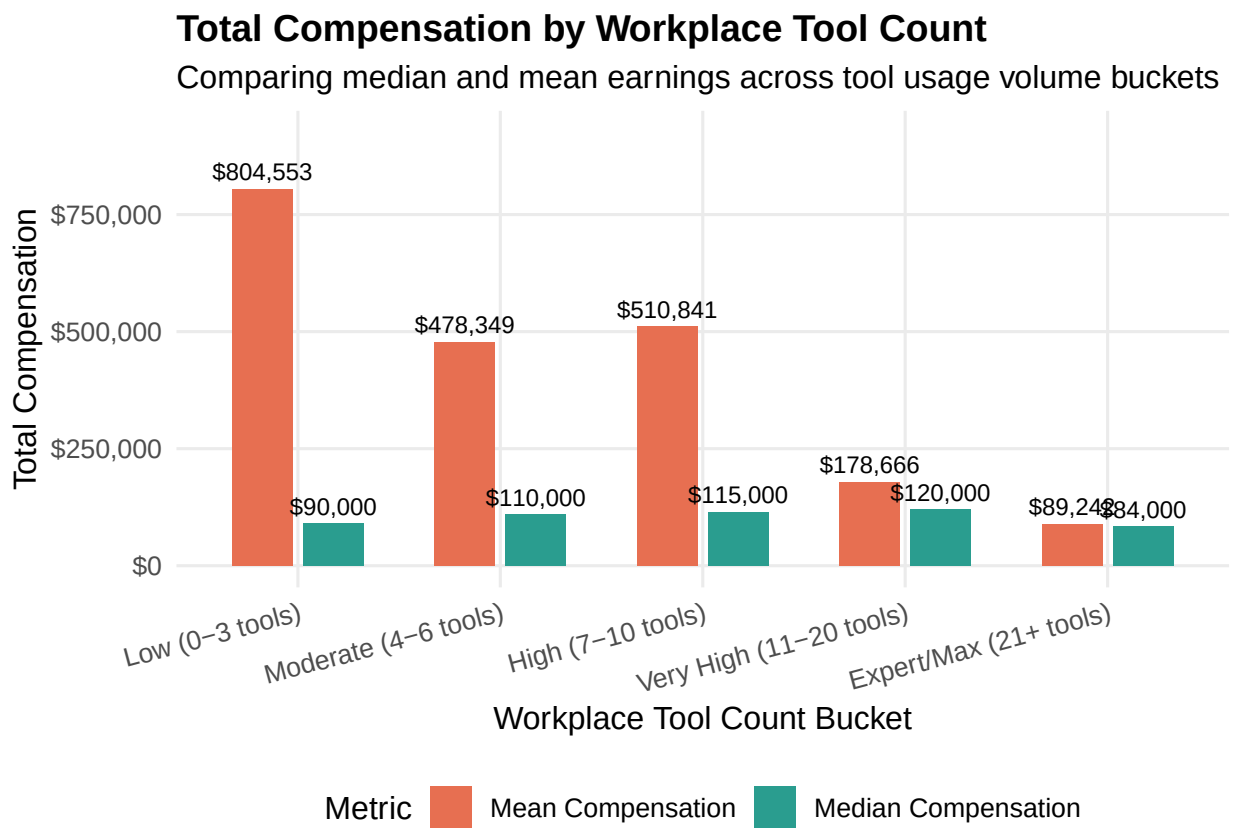
This chart shows median compensation for data, AI and engineering roles, with bar color showing the number of respondents. Engineering managers lead at $202,500, followed by database administrators ($200,000) and DevOps engineers ($144,700). Data engineers ($137,500), AI/ML engineers ($130,000) and applied scientists ($130,000) form the next tier.

Analysis-focused roles sit at the bottom: data scientists have a median of $70,000 and data or business analysts $54,000. Roles that build and maintain systems, such as DevOps, data engineering and AI/ML engineering, earn more than roles centered on analyzing data, and the highest-paid role is a management position. Cloud infrastructure engineers are an exception among the engineering roles, with a median of $85,000.



This chart compares median and mean compensation for each database used by at least 25 respondents. By median, cloud data platforms rank highest: BigQuery ($170,000), Snowflake ($150,000), Supabase ($144,400) and Cosmos DB ($140,000). The most widely used databases, PostgreSQL, MySQL, Microsoft SQL Server and SQLite, sit lower at $109,000 to $117,000, and MariaDB is lowest at $92,500.

The mean is higher than the median for every database, often by several hundred thousand dollars. MongoDB, for example, has a median of $119,000 but a mean of $772,091. A small number of very high reported values pull the averages up, so the median better reflects typical pay. Based on the median, specialized cloud and analytics databases are associated with higher pay than common general-purpose databases.



This chart groups respondents by the number of tools they use at work and compares median and mean compensation in each group. Median pay rises steadily as tool count increases, from $90,000 for Low (0-3 tools) to $110,000 for Moderate (4-6), $115,000 for High (7-10) and $120,000 for Very High (11-20). The Expert/Max group (21+ tools) breaks this pattern with the lowest median at $84,000, but it has only 14 respondents.

The mean is far above the median in the lower buckets, reaching $804,553 for Low and around $480,000 to $510,000 for Moderate and High, which shows a few very large reported values pulling those averages up. In the Very High and Expert/Max groups the mean and median are much closer ($178,666 vs. $120,000 and $89,242 vs. $84,000). Based on the median, working with a broader set of tools is associated with higher pay, up to about 20 tools.

```
knitr::opts_chunk$set(echo = TRUE)
```

Setup

Before anything runs, we're installing all the packages we need. Then we're defining where the data lives and create an output folder to save everything to. If the CSV hasn't been downloaded yet, we download it.

```
pkgs <- c("readr", "dplyr", "tidyr", "stringr", "ggplot2", "scales", "knitr")
missing <- pkgs[!vapply(pkgs, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing)) install.packages(missing, repos = "https://cloud.r-project.org")
invisible(lapply(pkgs, library, character.only = TRUE))

DATA_URL  <- "https://raw.githubusercontent.com/lhamo07/Project-03/main/data_final.csv"
```

```
DATA_FILE <- "data_final.csv"
OUT_DIR   <- "output"
dir.create(OUT_DIR, showWarnings = FALSE)
if (!file.exists(DATA_FILE)) download.file(DATA_URL, DATA_FILE, mode = "wb", quiet = TRUE)
```

Group definitions

We're sorting various different developer roles into different categories and storing them into variables ("DIFF", EXEC") and these roles can used with the different values of the "WORKING" variable.

```
DIFF <- c("Architect, software or solutions", "Data scientist", "AI/ML engineer",
          "Applied scientist", "Data engineer", "Cloud infrastructure engineer",
          "Engineering manager", "Senior executive (C-suite, VP, etc.)",
          "Founder, technology or otherwise")
EXEC <- c("Developer, full-stack", "Developer, back-end", "Developer, front-end",
          "Developer, desktop or enterprise applications", "Developer, mobile",
          "Developer, embedded applications or devices", "Developer, QA or test",
          "Developer, game or graphics")
WORKING <- c("Employed", "Independent contractor, freelancer, or self-employed")
```

Load data

We're reading into the CSV with R and storing the data into the raw variable.

```
raw <- read_csv(DATA_FILE, col_types = cols(.default = col_character()),
                locale = locale(encoding = "UTF-8"), show_col_types = FALSE)
```

Clean data

We're cleaning out the raw variable. We're filtering out any rows we don't need, removing duplicates, and converting compensation and job satisfaction from character to numeric. Then we narrow down to US-based, actively working people whose pay falls between $20K–$1M. After that, we split the DevType column (since one person can have multiple roles), map each role to a skill group, and then figure out each respondent's primary group — if they have even one Differentiating role, they're classified as Differentiating. Finally, we join that classification back to the main dataset and save both cleaned files.

```
n_raw <- nrow(raw)

df <- raw |>
  filter(str_detect(ResponseId, "^[0-9]+$")) |>
  distinct(ResponseId, .keep_all = TRUE) |>
  mutate(CompTotal = suppressWarnings(as.numeric(CompTotal)),
         JobSat    = suppressWarnings(as.numeric(JobSat)))
n_valid <- nrow(df)

df <- df |>
  filter(Employment %in% WORKING, !is.na(DevType),
         Country == "United States of America",
         between(CompTotal, 20000, 1000000))
n_us <- nrow(df)

roles_long <- df |>
```

```
  select(ResponseId, CompTotal, JobSat, DevType) |>
  separate_rows(DevType, sep = ";") |>
  mutate(Role = str_trim(DevType), DevType = NULL,
         SkillGroup = case_when(Role %in% DIFF ~ "Differentiating",
                                Role %in% EXEC ~ "Execution",
                                TRUE ~ NA_character_)) |>
  filter(!is.na(SkillGroup))

resp_group <- roles_long |>
  group_by(ResponseId) |>
  summarise(SkillGroup = if (any(SkillGroup == "Differentiating")) "Differentiating" else "Execution",
            .groups = "drop")
tidy <- df |> select(-DevType) |> inner_join(resp_group, by = "ResponseId")

write_csv(tidy,       file.path(OUT_DIR, "data_clean.csv"))
write_csv(roles_long, file.path(OUT_DIR, "data_roles_long.csv"))
```

```
tibble(Step = c("Raw rows", "Valid, unique ResponseId",
                "US working professionals with usable pay", "Classified into a skill group"),
       Respondents = c(n_raw, n_valid, n_us, nrow(tidy))) |>
  kable()
```

Step	Respondents
Raw rows	1036
Valid, unique ResponseId	1033
US working professionals with usable pay	191
Classified into a skill group	168

```
summary_tbl <- tidy |>
  group_by(SkillGroup) |>
  summarise(n = n(),
            median_comp = median(CompTotal),
            mean_comp   = round(mean(CompTotal)),
            mean_jobsat = round(mean(JobSat, na.rm = TRUE), 2),
            n_jobsat    = sum(!is.na(JobSat)),
            .groups = "drop")
write_csv(summary_tbl, file.path(OUT_DIR, "summary_by_group.csv"))
kable(summary_tbl, format.args = list(big.mark = ","))
```

SkillGroup	n	median_comp	mean_comp	mean_jobsat	n_jobsat
Differentiating	43	183,000	217,573	8.00	31
Execution	125	138,000	152,234	7.28	118

Plot theme

This is our visual theme for plots across our various graphs, defined once at the beginning.

```
COL <- c(Differentiating = "#2563c4", Execution = "#d9822b")
theme_proj <- theme_minimal(base_size = 12) +
  theme(panel.grid.minor = element_blank(),
        plot.title = element_text(face = "bold", color = "#1f2937"),
```

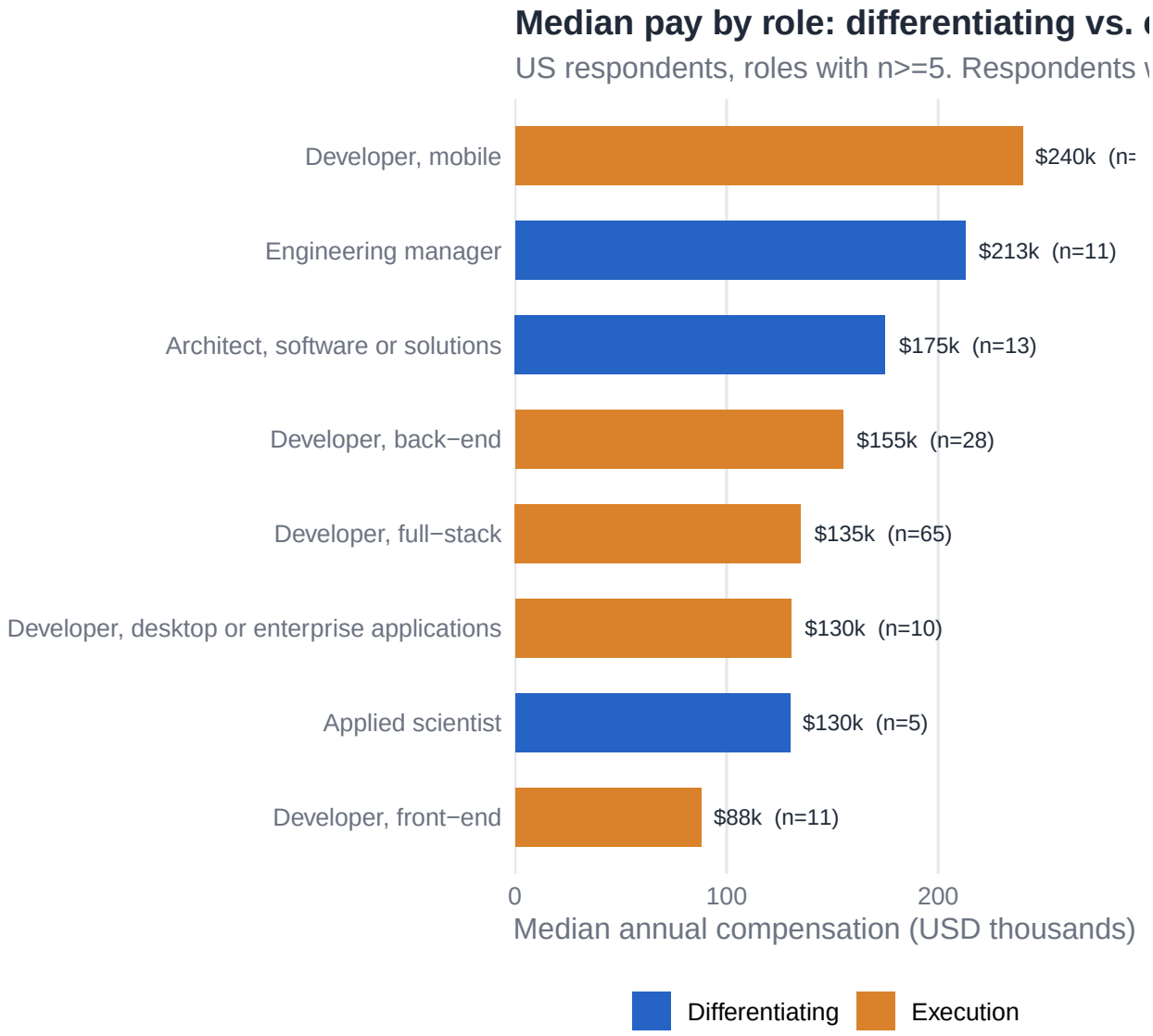
```
        plot.subtitle = element_text(color = "#6b7280"),
        axis.text = element_text(color = "#6b7280"),
        axis.title = element_text(color = "#6b7280"),
        legend.position = "bottom", legend.title = element_blank())
```

Graph 1

We're displaying a bar chart of median pay by individual role and skill group, filtered to roles with at least 5 respondents.

```
g1 <- roles_long |>
  group_by(Role, SkillGroup) |>
  summarise(median_comp = median(CompTotal), n = n(), .groups = "drop") |>
  filter(n >= 5)

p1 <- ggplot(g1, aes(median_comp / 1000, reorder(Role, median_comp), fill = SkillGroup)) +
  geom_col(width = 0.62) +
  geom_text(aes(label = sprintf("$%.0fk  (n=%d)", median_comp / 1000, n)),
            hjust = -0.1, size = 3.3, color = "#1f2937") +
  scale_fill_manual(values = COL) +
  scale_x_continuous(expand = expansion(mult = c(0, 0.22))) +
  labs(title = "Median pay by role: differentiating vs. execution",
       subtitle = "US respondents, roles with n>=5. Respondents with several roles appear in each.",
       x = "Median annual compensation (USD thousands)", y = NULL) +
  theme_proj +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.major.x = element_line(color = "#e5e7eb"))
ggsave(file.path(OUT_DIR, "graph1_comp_by_role.png"), p1,
       width = 9.5, height = 6, dpi = 180, bg = "white")
p1
```



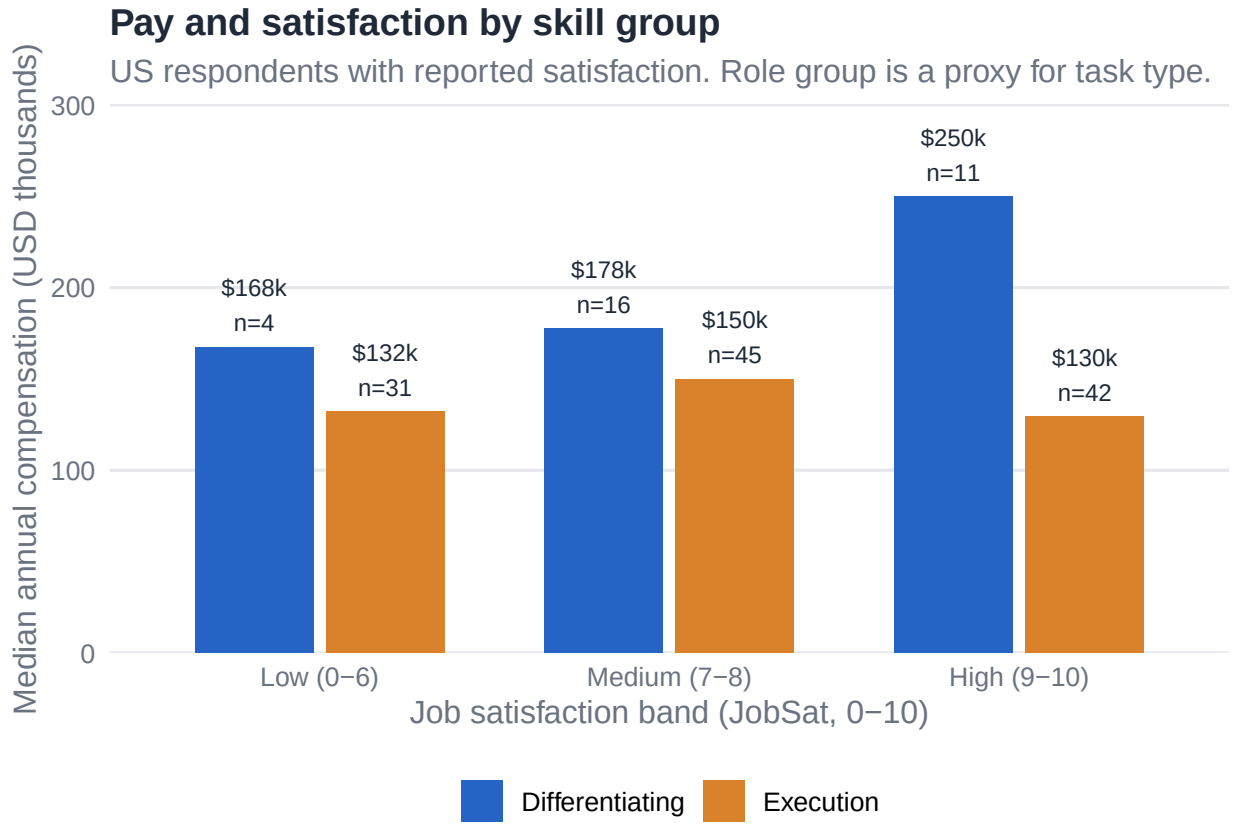
Graph 2

We're displaying a grouped bar chart compares pay across job satisfaction levels (Low, Medium, High) for both skill groups side by side.

```
g2 <- tidy |>
  filter(!is.na(JobSat)) |>
  mutate(Band = cut(JobSat, c(-Inf, 6, 8, Inf),
                    labels = c("Low (0-6)", "Medium (7-8)", "High (9-10)"))) |>
  group_by(Band, SkillGroup) |>
  summarise(median_comp = median(CompTotal), n = n(), .groups = "drop")

p2 <- ggplot(g2, aes(Band, median_comp / 1000, fill = SkillGroup)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.68) +
  geom_text(aes(label = sprintf("$%.0fk\nn=%d", median_comp / 1000, n)),
            position = position_dodge(width = 0.75), vjust = -0.3,
```

```
            size = 3.2, color = "#1f2937") +
  scale_fill_manual(values = COL) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.2))) +
  labs(title = "Pay and satisfaction by skill group",
       subtitle = "US respondents with reported satisfaction. Role group is a proxy for task type.",
       x = "Job satisfaction band (JobSat, 0-10)",
       y = "Median annual compensation (USD thousands)") +
  theme_proj +
  theme(panel.grid.major.y = element_line(color = "#e5e7eb"),
        panel.grid.major.x = element_blank())
ggsave(file.path(OUT_DIR, "graph2_satisfaction_vs_comp.png"), p2,
       width = 9, height = 5.5, dpi = 180, bg = "white")
p2
```



Session info

A log of our R environment details.

```
writeLines(capture.output(sessionInfo()), file.path(OUT_DIR, "sessionInfo.txt"))
sessionInfo()
```

```
## R version 4.5.3 (2026-03-11)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
```

```
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] ggrepel_0.9.8   scales_1.4.0    knitr_1.52      lubridate_1.9.5
##  [5] forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1     purrr_1.2.2
##  [9] readr_2.2.0     tidyr_1.3.2     tibble_3.3.1    ggplot2_4.0.3
## [13] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     stringi_1.8.9      hms_1.1.4          digest_0.6.39
##  [5] magrittr_2.0.5     evaluate_1.0.5     grid_4.5.3         timechange_0.4.0
##  [9] RColorBrewer_1.1-3 fastmap_1.2.0      viridisLite_0.4.3  textshaping_1.0.5
## [13] cli_3.6.6          rlang_1.3.0        crayon_1.5.3       bit64_4.8.6
## [17] withr_3.0.3        yaml_2.3.12        otel_0.2.0         tools_4.5.3
## [21] parallel_4.5.3     tzdb_0.5.0         curl_8.0.0         vctrs_0.7.3
## [25] R6_2.6.1           lifecycle_1.0.5    bit_4.6.0          vroom_1.7.1
## [29] ragg_1.5.2         pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
## [33] glue_1.8.1         Rcpp_1.1.2         systemfonts_1.3.2  xfun_0.61
## [37] tidyselect_1.2.1   rstudioapi_0.19.0  farver_2.1.2       htmltools_0.5.9
## [41] rmarkdown_2.32     labeling_0.4.3     compiler_4.5.3     S7_0.2.2
```